FEU INSTITUTE OF TECHNOLOGY

COLLEGE OF COMPUTER STUDIES AND MULTIMEDIA ARTS

CS0065
(INTELLIGENT SYSTEMS)

TECHNICAL ASSESSMENT

2

RULE – BASED AGENT

Student Name	Joshua Emmanuel A. Guerrero
Section:	AN41
Professor:	Ms. Tan
Date:	10/02/2026

I. BACKGROUND INFORMATION

A Rule-Based Agent is a type of intelligent agent that operates by following predefined rules to decide its actions. In this lab, students will work with a vacuum cleaner agent that can clean rooms or move between rooms depending on their state (dirty or clean).

The goal is to understand how agents perceive the environment, act according to rules, and update the environment.

II. TECHNICAL ASSESSMENT

RULE – BASED AGENT

Objective

At the end of this laboratory activity, students are expected to:

1. Explain the concept of rule-based agents.
2. Implement a rule-based agent in Python.
3. Interact with an environment by perceiving and acting based on rules.
4. Observe and analyze the behavior of an agent in a controlled environment.

Materials Needed

- Computer with Python 3.x installed
- Text editor or IDE (e.g., VSCode, PyCharm)
- Optional: Jupyter Notebook for interactive testing

General Instructions

1. This **Technical Assessment (TA4)** must be completed individually.
2. Follow the Task below.
3. Include clear comments in your code to describe its logic.
4. Each program should display the correct input, process, and output.
5. Ensure that your script displays clear and readable outputs.
6. Compile all Python codes and corresponding outputs into one PDF file.
7. Save it as **TA2_lastname.pdf**
   (Example: TA2_Patilano.pdf)
8. Submit the **python file** and **PDF** file.

Laboratory Task

Pre-Lab Preparation

1. Review the Python concepts:
   - Classes and objects
   - If-Else statements

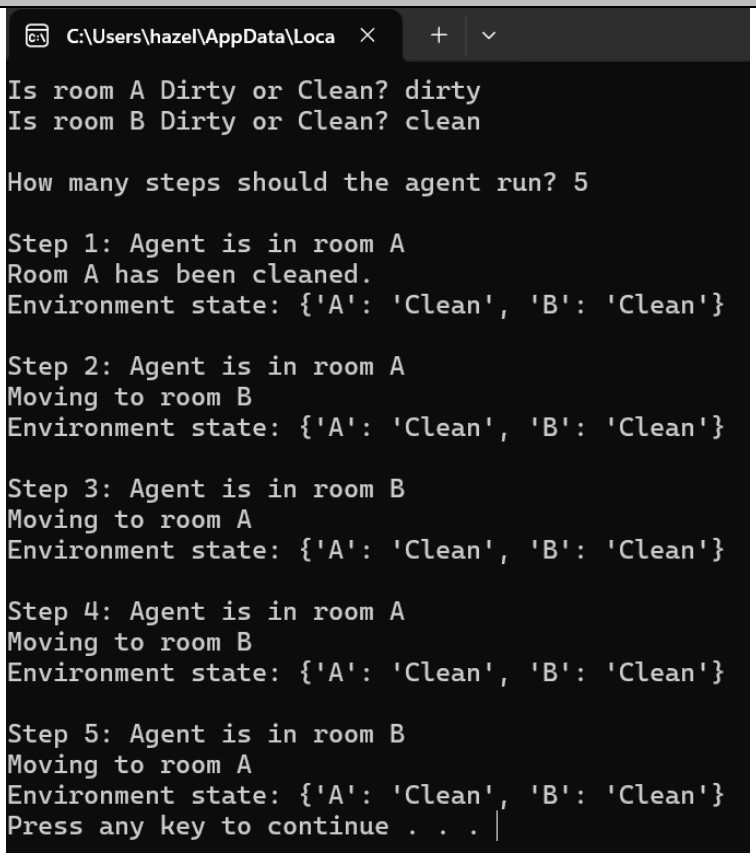
- Loops (for loop)
- Functions / methods

2. Understand the environment:
   - Two rooms: A and B
   - Each room can be Dirty or Clean

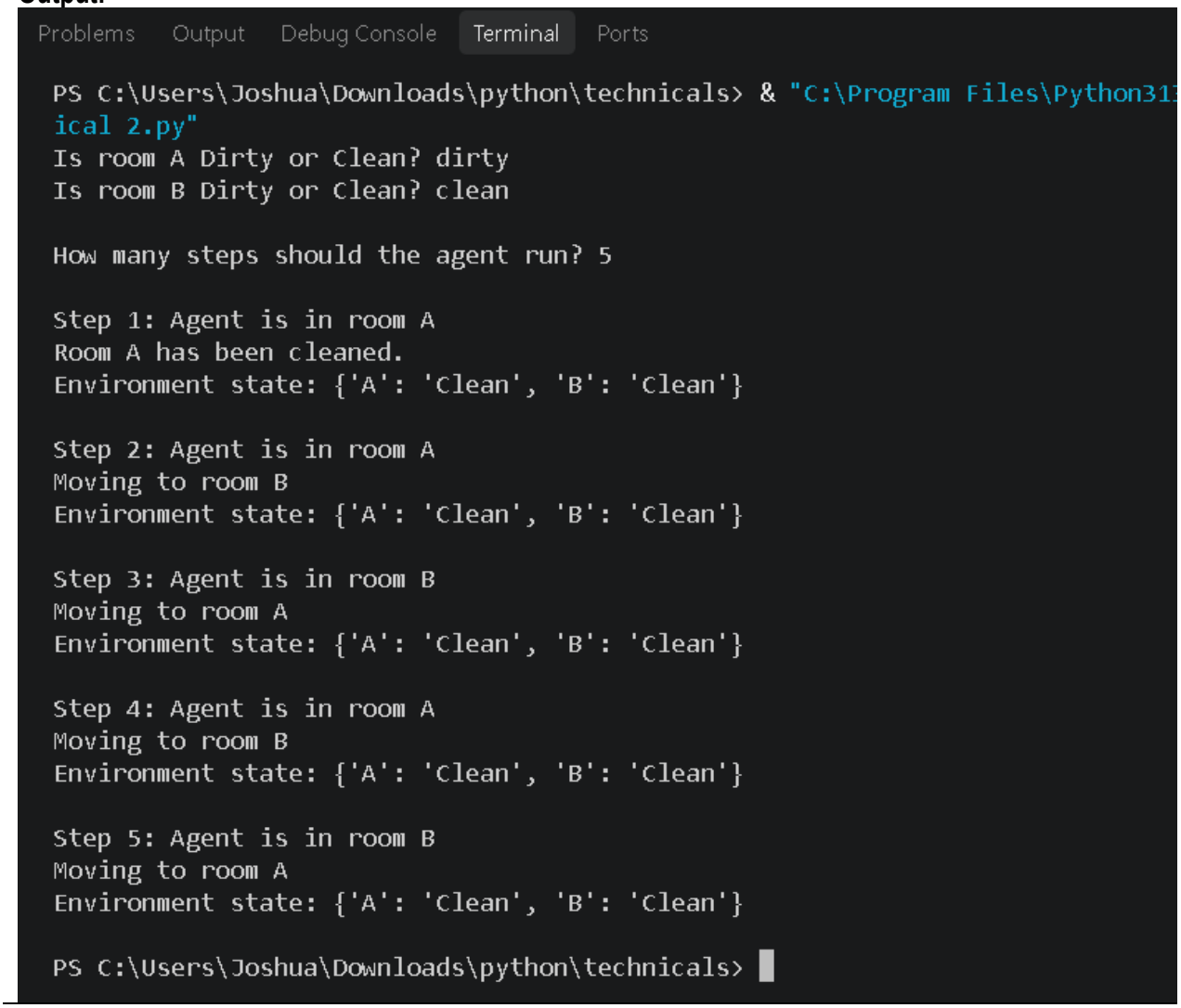
Task 1: Set Up the Environment

1. Initialize two rooms using a Python dictionary.
2. Allow the user to set the **initial state** of each room (dirty or clean).
3. Write a function to **check if a room is dirty** and a function to **clean a room**.

Sample Output



Output:



Task 2: Implement the Rule-Based Agent

1. Create a class for the agent.
2. Implement the **perceive-and-act method**:
    - Rule 1: If the current room is dirty, clean it.
    - Rule 2: If the current room is clean, move to the other room.
3. Implement a **move method** to switch rooms.

Output:

```
PS C:\Users\Joshua\Downloads\python\technicals> & "C:\Program Files\Python3
ical 2.py"
Is room A Dirty or Clean? clean
Is room B Dirty or Clean? clean

How many steps should the agent run? 5

Step 1: Agent is in room A
Moving to room B
Environment state: {'A': 'Clean', 'B': 'Clean'}

Step 2: Agent is in room B
Moving to room A
Environment state: {'A': 'Clean', 'B': 'Clean'}

Step 3: Agent is in room A
Moving to room B
Environment state: {'A': 'Clean', 'B': 'Clean'}

Step 4: Agent is in room B
Moving to room A
Environment state: {'A': 'Clean', 'B': 'Clean'}

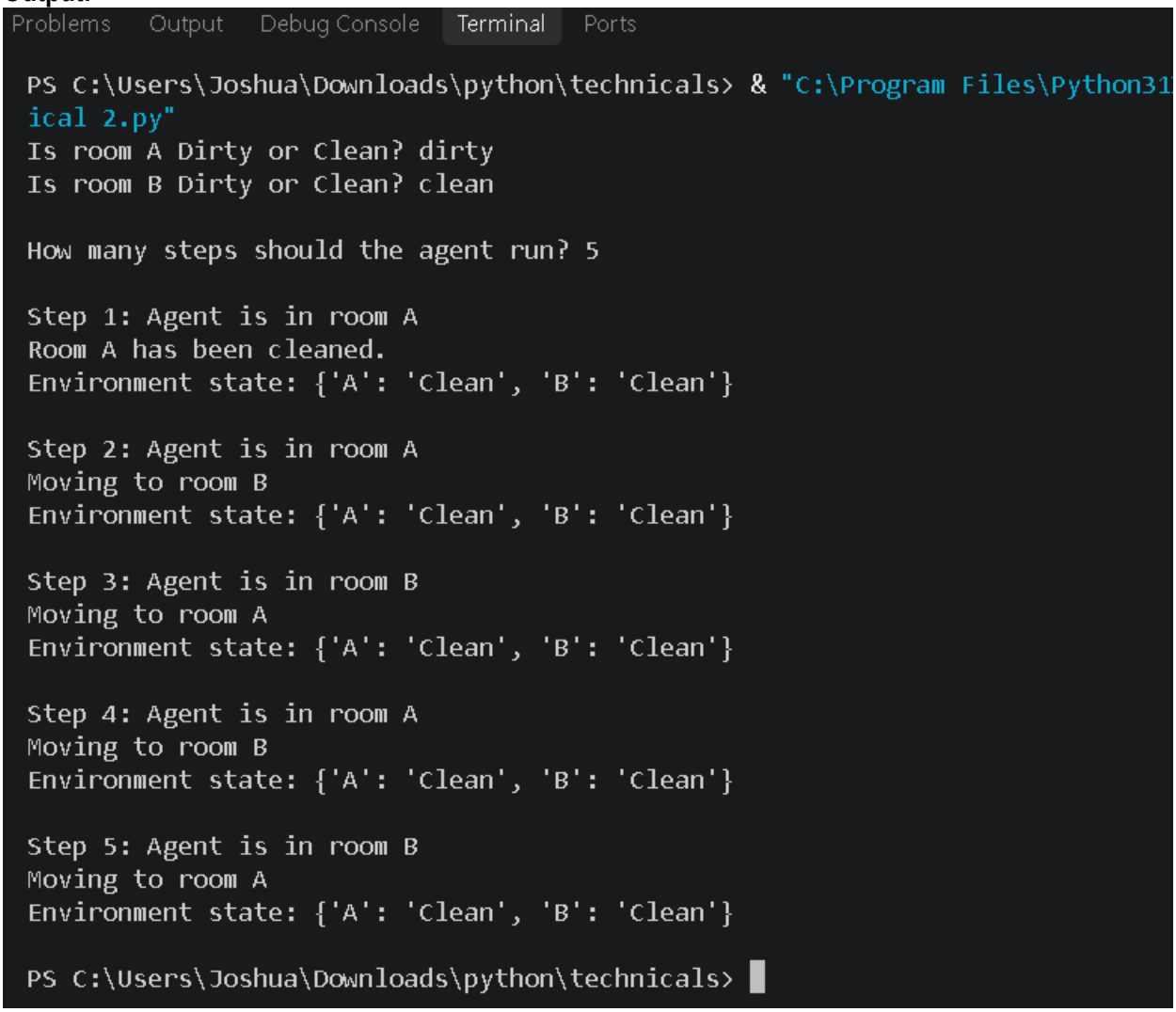
Step 5: Agent is in room A
Moving to room B
Environment state: {'A': 'Clean', 'B': 'Clean'}

PS C:\Users\Joshua\Downloads\python\technicals>
```

Task 3: Run the Simulation

1. Ask the user how many steps the simulation should run.
2. Run the agent in a loop for the specified steps.
3. After each step, print:
    - The agent's current location
    - The action taken (cleaned/moved)
    - Current state of the environment

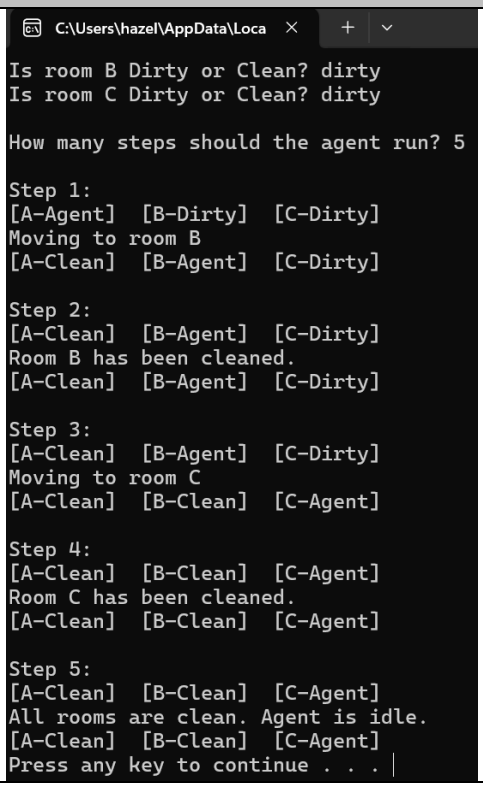
Output:


Problems
Output
Debug Console
Terminal
Ports
PS C:\Users\Joshua\Downloads\python\technicals> & "C:\Program Files\Python31
ical 2.py"
Is room A Dirty or Clean? dirty
Is room B Dirty or Clean? clean
How many steps should the agent run? 5
Step 1: Agent is in room A
Room A has been cleaned.
Environment state: {'A': 'Clean', 'B': 'Clean'}
Step 2: Agent is in room A
Moving to room B
Environment state: {'A': 'Clean', 'B': 'Clean'}
Step 3: Agent is in room B
Moving to room A
Environment state: {'A': 'Clean', 'B': 'Clean'}
Step 4: Agent is in room A
Moving to room B
Environment state: {'A': 'Clean', 'B': 'Clean'}
Step 5: Agent is in room B
Moving to room A
Environment state: {'A': 'Clean', 'B': 'Clean'}
PS C:\Users\Joshua\Downloads\python\technicals>

Optional (Bonus Task)

- Add a **third room (C)** and update the agent logic.
- Make the agent **randomly choose which dirty room to clean next**.
- Visualize the agent moving between rooms using a **text-based grid**.

Sample Output (Bonus Task)



Output:

```
PS C:\Users\Joshua\Downloads\python\technicals> & "C:\Prog
ical 2.py"
Is room A Dirty or Clean? clean
Is room B Dirty or Clean? dirty
Is room C Dirty or Clean? dirty

How many steps should the agent run? 5

Step 1:
[A-Agent]  [B-Dirty]  [C-Dirty]
Moving to room C
[A-Clean]  [B-Dirty]  [C-Agent]

Step 2:
[A-Clean]  [B-Dirty]  [C-Agent]
Room C has been cleaned.
[A-Clean]  [B-Dirty]  [C-Agent]

Step 3:
[A-Clean]  [B-Dirty]  [C-Agent]
Moving to room B
[A-Clean]  [B-Agent]  [C-Clean]

Step 4:
[A-Clean]  [B-Agent]  [C-Clean]
Room B has been cleaned.
[A-Clean]  [B-Agent]  [C-Clean]

Step 5:
[A-Clean]  [B-Agent]  [C-Clean]
All rooms are clean. Agent is idle.
[A-Clean]  [B-Agent]  [C-Clean]

PS C:\Users\Joshua\Downloads\python\technicals>
```

Your Code - Laboratory Tasks ⇩

Laboratory Task 1-3

```
# TASK 1: SET UP THE ENVIRONMENT

def is_dirty(environment, room):
    """Checks if a room is dirty"""
    return environment[room] == "Dirty"
```

```
def clean_room(environment, room):
    """Cleans the specified room"""
    environment[room] = "Clean"

# TASK 2: IMPLEMENT THE RULE-BASED AGENT

class VacuumAgent:

    def __init__(self, start_room="A"):
        self.location = start_room

    def move(self):
        """Switches location between Room A and Room B"""
        if self.location == "A":
            self.location = "B"
        else:
            self.location = "A"

    def perceive_and_act(self, environment):
        current_room = self.location

        # Rule 1: If current room is dirty, clean it
        if is_dirty(environment, current_room):
            clean_room(environment, current_room)
            return f"Room {current_room} has been cleaned."

        # Rule 2: If current room is clean, move to the other room
        else:
            self.move()
            return f"Moving to room {self.location}"

# TASK 3: RUN THE SIMULATION

def main():
    # Ask user for initial states
    room_a = input("Is room A Dirty or Clean? ").strip().capitalize()
    room_b = input("Is room B Dirty or Clean? ").strip().capitalize()

    # Create environment dictionary (Task 1)
    environment = {"A": room_a, "B": room_b}

    print()
    steps = int(input("How many steps should the agent run? "))
```

```
    print()

    # Create agent instance (Task 2)
    agent = VacuumAgent(start_room="A")

    # Run simulation loop
    for step in range(1, steps + 1):
        print(f"Step {step}: Agent is in room {agent.location}")
        action_message = agent.perceive_and_act(environment)
        print(action_message)
        print(f"Environment state: {environment}")
        print()

if __name__ == "__main__":
    main()
```

BONUS TASK CODE:

```
import random

# ========================================
# BONUS TASK: 3-ROOM ENVIRONMENT & AGENT
# ========================================

def is_dirty(environment, room):
    """Checks if a room is dirty"""
    return environment[room] == "Dirty"

def clean_room(environment, room):
    """Cleans the specified room"""
    environment[room] = "Clean"

def print_grid(environment, agent_location):
    """Prints the text-based grid visualization"""
    grid_items = []
    for room in ["A", "B", "C"]:
        # If the agent is in this room, display [Room-Agent]
        if room == agent_location:
            grid_items.append(f"[{room}-Agent]")
        else:
            # Otherwise display [Room-State] (e.g., [A-Clean] or [B-Dirty])
            grid_items.append(f"[{room}-{environment[room]}]")
```

```
    print("  ".join(grid_items))

class BonusVacuumAgent:

    def __init__(self, start_room="A"):
        self.location = start_room

    def perceive_and_act(self, environment):
        current_room = self.location

        # Rule 1: If current room is dirty, clean it
        if is_dirty(environment, current_room):
            clean_room(environment, current_room)
            return f"Room {current_room} has been cleaned."

        # Rule 2: If current room is clean, randomly choose a dirty room to move to
        dirty_rooms = [
            room for room in environment if is_dirty(environment, room)
        ]

        if dirty_rooms:
            # Randomly select one dirty room
            next_room = random.choice(dirty_rooms)
            self.location = next_room
            return f"Moving to room {self.location}"
        else:
            # If all rooms are clean
            return "All rooms are clean. Agent is idle."

def main_bonus():
    # Take initial room inputs (Room A defaults to clean in sample output or we can ask)
    room_a = input("Is room A Dirty or Clean? ").strip().capitalize()
    room_b = input("Is room B Dirty or Clean? ").strip().capitalize()
    room_c = input("Is room C Dirty or Clean? ").strip().capitalize()

    environment = {"A": room_a, "B": room_b, "C": room_c}

    print()
    steps = int(input("How many steps should the agent run? "))
    print()

    agent = BonusVacuumAgent(start_room="A")

    for step in range(1, steps + 1):
```

```
        print(f"Step {step}:")

        # 1. Show grid state BEFORE action
        print_grid(environment, agent.location)

        # 2. Perform action
        action_message = agent.perceive_and_act(environment)
        print(action_message)

        # 3. Show grid state AFTER action
        print_grid(environment, agent.location)
        print()

if __name__ == "__main__":
    main_bonus()
```

Task 4: Analysis Questions

After running the simulation, answer the following:

1. What rules does the agent follow?
2. How does the agent decide when to clean and when to move?
3. What happens if both rooms are already clean at the start?
4. How would you modify the agent to handle **more than two rooms**?

Answer the Questions ⇩

1. - The agent operates using simple condition-action rules based on its current location. Rule 1 states that if the room the agent is currently in is dirty, it cleans the room. Rule 2 states that if the current room is clean, it moves to the other room.
2. - The decision is purely perception-driven and based on the state of its current room. At each step, it checks the environment dictionary to see if its location is marked as dirty. If it's dirty, the cleaning function is triggered; otherwise, it defaults to executing the movement method.
3. - If both rooms start as clean, Rule 1 is never triggered because neither room ever evaluates to dirty. As a result, Rule 2 will continuously execute, causing the agent to endlessly ping-pong back and forth between Room A and Room B without taking any cleaning actions.
4. - To support more rooms, I would update the environment structure to store additional room keys and change the movement logic so it isn't limited to a simple binary switch. The agent could either track an ordered list of adjacent rooms to navigate sequentially or query the environment for a list of all dirty rooms and randomly pick one to move to, just like we did in the bonus task